

Task 6: Introduction to Cryptography

Symmetric Encryption:

Symmetric encryption uses a single, shared secret key for both encrypting (scrambling) and decrypting (unscrambling) data, making it fast and efficient for bulk data, but requiring secure key exchange. Also known as secret-key or private-key encryption, it transforms readable plaintext into unreadable ciphertext, protecting sensitive information like database records or internal network traffic, with [AES](#) being a common algorithm.

Asymmetric Encryption:

Asymmetric encryption, or [public-key cryptography](#), uses a pair of mathematically linked keys—a public key for encryption and a private key for decryption—allowing secure communication without sharing secrets; the public key encrypts, and only the corresponding private key can decrypt, solving the key exchange problem of [symmetric encryption](#) but is slower, often used in hybrid systems (like HTTPS/TLS) for secure key exchange, digital signatures, and authenticating users.

Create a Sample File:

```
(hari@vbox)-[~]  
$ echo "This is a cryptography demo file" > secret.txt  
  
(hari@vbox)-[~]  
$ cat secret.txt  
This is a cryptography demo file
```

1) Encrypt File Using AES (Symmetric Encryption):

```
(hari@vbox)-[~]  
$ openssl enc -aes-256-cbc -salt -in secret.txt -out secret.enc  
enter AES-256-CBC encryption password:  
Verifying - enter AES-256-CBC encryption password:  
*** WARNING : deprecated key derivation used.  
Using -iter or -pbkdf2 would be better.
```

Check encrypted file:

```
(hari@vbox)-[~]  
$ cat secret.enc  
Salted__  
J+l<l·%rN9K  
2J,  
aM:
```

Output will be unreadable (ciphertext).

Decrypt AES file:

```
(hari@vbox)-[~]  
$ openssl enc -aes-256-cbc -d -in secret.enc -out secret_decrypted.txt  
enter AES-256-CBC decryption password:  
*** WARNING : deprecated key derivation used.  
Using -iter or -pbkdf2 would be better.
```

```
(hari@vbox)-[~]  
$ cat secret_decrypted.txt  
This is a cryptography demo file
```

AES uses the same secret key for encryption and decryption and is fast and efficient.

2) Generate RSA Keys (Asymmetric Encryption):

Generate Private Key:

```
(hari@vbox)-[~]  
$ openssl genrsa -out private.key 2048
```

Generate Public Key:

```
(hari@vbox)-[~]  
$ openssl rsa -in private.key -pubout -out public.key  
writing RSA key
```

View keys:

```
(hari@vbox)-[~]  
$ ls -l *.key  
-rw----- 1 hari hari 1704 Jan 24 22:45 private.key  
-rw-rw-r-- 1 hari hari  451 Jan 24 22:46 public.key
```

Encrypt & Decrypt Using RSA:

Encrypt using Public Key:

```
(hari@vbox)-[~]  
$ openssl rsautl -encrypt -inkey public.key -pubin -in secret.txt -out rsa_encrypted.bin  
The command rsautl was deprecated in version 3.0. Use 'pkeyutl' instead.
```

Decrypt using Private Key:

```
(hari@vbox)-[~]  
$ openssl rsautl -decrypt -inkey private.key -in rsa_encrypted.bin -out rsa_decrypted.txt  
The command rsautl was deprecated in version 3.0. Use 'pkeyutl' instead.
```

```
(hari@vbox)-[~]  
$ cat rsa_decrypted.txt  
This is a cryptography demo file
```

Only private key holder can decrypt data encrypted with public key.

3) Digital Signatures (Integrity + Authenticity):

Create a signature:

```
(hari@vbox)-[~]  
$ openssl dgst -sha256 -sign private.key -out signature.bin secret.txt
```

Verify signature:

```
(hari@vbox)-[~]  
$ openssl dgst -sha256 -verify public.key -signature signature.bin secret.txt  
Verified OK
```

Digital signatures ensure data integrity and sender authenticity.

4) Hash Files & Verify Integrity:

Generate hash:

```
(hari@vbox)-[~]  
$ sha256sum secret.txt > hash.txt
```

```
(hari@vbox)-[~]  
$ cat hash.txt  
e72c330c961a934eb0fe6d3f357b378dd6d8e8c7fd4c744912015983241d55d4 secret.txt
```

Verify later:

```
(hari@vbox)-[~]  
$ sha256sum -c hash.txt  
secret.txt: OK
```

Hashing ensures file integrity and detects unauthorized changes.

Compare Encryption Algorithms (Table):

Algorithm	Type	Speed	Use Case
AES	Symmetric	Fast	File & disk encryption
RSA	Asymmetric	Slow	Key exchange
SHA-256	Hash	Very fast	Integrity check

Real-World Usage:

HTTPS:

- Uses RSA for key exchange
- Uses AES for data encryption
- Uses certificates for trust

VPN:

- Encrypts network traffic
- Protects data over public Wi-Fi

Banking Apps:

- AES for data
- RSA for authentication
- Hashing for passwords